

Kubernetes Architecture and Control Plane Specification

Section 1: Control Plane Overview

The Kubernetes control plane manages worker nodes and the pods in the cluster.

The control plane components make global decisions about the cluster (for example, scheduling), as well as detecting and responding to cluster events.

Key components include:

1. kube-apiserver: The API server is the front end for the Kubernetes control plane.

It exposes the Kubernetes API and scales horizontally.

2. etcd: Consistent and highly-available key value store used as Kubernetes backing store.

3. kube-scheduler: Watches for newly created Pods with no assigned node, and selects a node.

4. kube-controller-manager: Runs controller processes (Node controller, Job controller).

Workloads, Pods, and Service Networking

Section 2: Pod Lifecycle and ReplicaSets

Pods are the smallest deployable units of computing that can be created and managed in Kubernetes.

A Pod encapsulates one or more containers, shared storage resources, and unique network IP.

A ReplicaSet ensures that a specified number of pod replicas are running at any given time.

However, Deployments are high-level abstractions that manage ReplicaSets declaratively.

Section 3: ClusterIP and Headless Services

The default Service type is ClusterIP, which exposes the Service on a cluster-internal IP.

Headless services are defined by explicitly specifying None for clusterIP (.spec.clusterIP: None).

They allow direct pod-to-pod communication without a proxy.

Cluster Security, RBAC, and Network Policies

Section 4: Role-Based Access Control (RBAC)

Role-Based Access Control uses the `rbac.authorization.k8s.io` API group to drive authorization decisions.

A Role always sets permissions within a particular namespace; ClusterRole applies cluster-wide.

Subjects can be Users, Groups, or ServiceAccounts.

Section 5: Network Policies and Zero Trust

By default, pods are non-isolated and accept traffic from any source.

NetworkPolicies specify how groups of pods are allowed to communicate with each other and other network endpoints.

NetworkPolicy resources use podSelector, ingress rules, and egress rules to isolate pod traffic.